

Book2Screen: A Web-Based Relational Database and Review Platform for Literary-to-Cinematic Adaptations

Elif Kuş[✉]

Department of Computer Engineering
Fenerbahçe University
Istanbul, Turkey

Abstract—The cinematic adaptation of literary works represents a significant cultural and commercial intersection. However, the systematic mapping and analysis of the relationships between source materials and their film counterparts lack a dedicated, structurally rigorous platform. This paper presents the design, implementation, and deployment of Book2Screen, a comprehensive web application engineered to bridge this gap. The platform is built upon a normalized relational database model that enforces strict ontological links between literary works (books) and their cinematic adaptations (movies), encapsulating metadata such as authorship, directorial vision, cast performance, and commercial metrics. Developed using a full-stack architecture with ASP.NET Core 7.0 and PostgreSQL, the system features a responsive user interface, real-time search functionality, and user interaction mechanisms for reviews and ratings. The application is containerized and deployed on Microsoft Azure App Service, ensuring scalability and global accessibility. This project demonstrates a practical implementation of modern web development paradigms to create a specialized knowledge base for adaptation studies.

Keywords— Web Application, Relational Database, Literary Adaptation, ASP.NET Core, PostgreSQL, Cloud Deployment.

I. INTRODUCTION

The cinematic adaptation of literary works represents a complex interplay between narrative traditions and visual media. While individual platforms exist for reviewing books or films independently, there remains a significant gap in digital tools that systematically examine the transformative process from source material to screen adaptation. Current solutions require researchers, critics, and enthusiasts to manually cross-reference disparate sources such as IMDb for film metadata and Goodreads for literary analysis, a process that is both time-consuming and prone to inconsistencies in data correlation.

Book2Screen addresses this fundamental need by establishing a unified digital ecosystem specifically engineered to model, query, and analyze the adaptive relationship between literature and cinema. The platform transcends mere cataloging by implementing a relational database architecture that explicitly codifies the connections between source texts and their cinematic interpretations. This approach enables structured exploration of adaptation patterns, directorial interpretations, narrative transformations, and audience reception across media boundaries. By serving simultaneously as an archival repository and an interactive social platform, Book2Screen

facilitates both academic research and community engagement with adaptation studies.

II. SYSTEM ARCHITECTURE AND TECHNOLOGY STACK

The Book2Screen application implements a five-layer architecture based on software engineering principles of separation of concerns, modularity, and scalability. This architecture ensures maintainability while supporting future enhancements.

- **User Interface Layer (Frontend):** Implemented using HTML5, CSS3, and vanilla JavaScript with Bootstrap 5 framework for responsive design. The interface provides consistent user experience across devices, with a custom "Dark Mode" aesthetic for comfortable media browsing.
- **Presentation Layer (ASP.NET Core MVC):** Developed with ASP.NET Core 7.0 MVC framework, this layer manages HTTP/HTTPS requests and responses. It separates data models, business logic, and presentation concerns, supporting both server-side rendering and API endpoints.
- **Business Logic Layer (Domain Services):** Implements domain-specific logic through Service Pattern architecture. Service classes handle adaptation analysis rules, similarity scoring, and user content moderation, keeping business rules independent of presentation and data access details.
- **Data Access Layer (Repository Pattern):** Uses Unit of Work and Repository patterns with Entity Framework Core 7.0 ORM. This layer abstracts database operations, ensures transactional integrity, and translates between C# models and PostgreSQL tables.
- **Infrastructure Layer (Persistence):** Employs PostgreSQL as the relational database management system for its SQL standards compliance, ACID properties, and advanced features. The schema uses referential integrity constraints and optimized indexing strategies.
- **Cloud Deployment:** The application is deployed on Microsoft Azure. The web component runs on Azure App Service with auto-scaling and continuous deployment. The database uses Azure Database for PostgreSQL with automated backups, high availability, and security features.

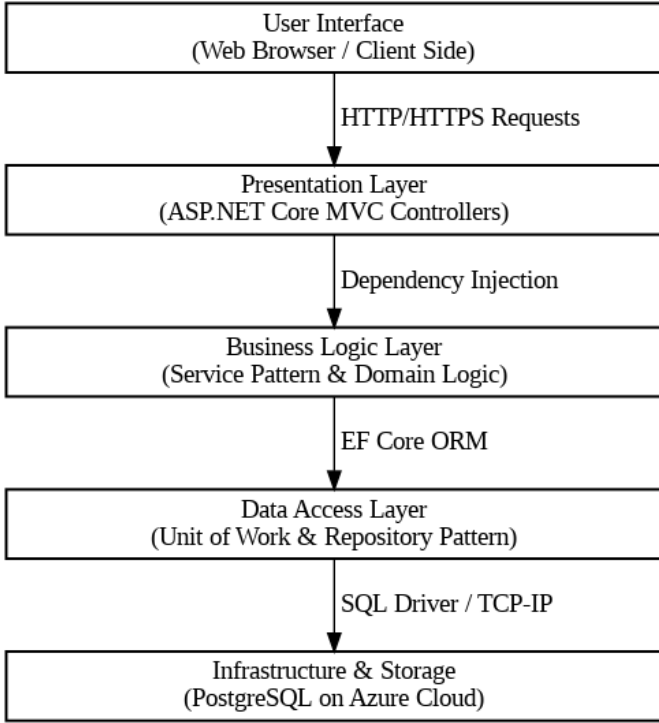


Figure 1: Five-layer architecture of the Book2Screen application showing data flow from user interface to persistent storage.

III. DATABASE DESIGN AND RELATIONAL DATA MODEL

The core intellectual contribution of Book2Screen lies in its data model, which rigorously defines the entities and relationships central to adaptation studies.

A. Entity Design

Key entities include Author, Book, Director, Movie, Actor, Genre, and User. Each entity possesses a defined set of attributes (e.g., Book has PublicationYear, PageCount; Movie has BoxOfficeRevenue, IMDbRating).

B. Relationship Mapping

The model enforces critical relational constraints:

- One-to-Many: An Author can write many Books. A Director can direct many Movies.
- One-to-One (with Foreign Key): A Movie is linked to its source Book via a mandatory BookId foreign key, establishing the fundamental adaptation link.
- Many-to-Many: Implemented through junction tables (MovieActor, MovieGenre, BookGenre) to model the relationships between Movies and Actors, and entities with Genres.

C. Implementation

The database schema was realized using the Code-First development approach with Entity Framework Core 7.0. This methodology allowed for model definition in C# classes, with the database schema being generated and managed via migrations.

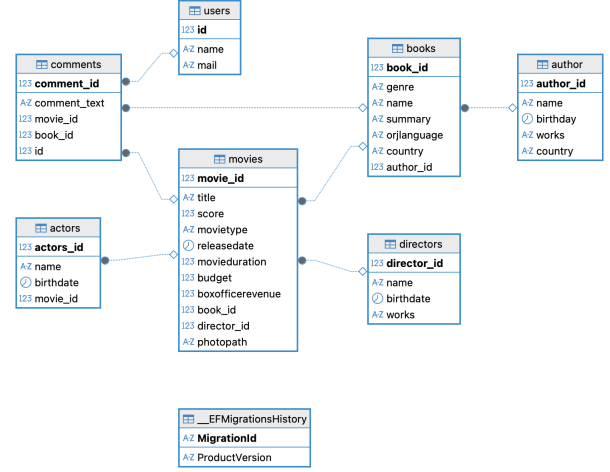


Figure 2: Entity-Relationship Diagram (ERD) illustrating the normalized schema of the Book2Screen database.

Table I: Database Entities and Their Key Attributes

Entity	Key Attributes	Purpose
Book	Title, ISBN, AuthorId, PageCount, PublicationYear	Source literary work
Movie	Title, DirectorId, BookId, Duration, BoxOffice, IMDbRating	Cinematic adaptation
Author	Name, Biography, Nationality, BirthDate	Creator of source material
Director	Name, Biography, Film-Count, Nationality	Creator of adaptation
Actor	Name, BirthDate, Nationality, Biography	Performer in adaptation
Review	UserId, MovieId, Rating, Content, CreatedDate	User-generated analysis
Genre	Name, Description	Category classification
User	Username, Email, PasswordHash, RegistrationDate	Platform user account

IV. IMPLEMENTATION AND USER INTERFACE DESIGN

A. Data Acquisition and Curation

Initial dataset population involved a hybrid methodology. Structured data (e.g., cast lists, runtime, box office figures) was programmatically fetched from public APIs such as IMDb and Goodreads where permissible. This was supplemented with manually curated data for niche adaptations and to ensure data quality and relational integrity.

B. Frontend Component Design

- Catalog Views: The Movie and Book catalogs are presented using Bootstrap card components within a responsive grid. CSS3 hover effects provide immediate visual feedback.
- Dynamic Search: A client-side JavaScript search engine performs real-time filtering on the catalog based on user input, enhancing usability.

- Detail Pages: Each major entity (Author, Director, Actor, Book, Movie) has a dedicated detail page, dynamically generated using Razor views and populated with related data via model binding.

C. Core Functionality

- User Authentication: A secure registration and login system, utilizing ASP.NET Core Identity, gates user-generated content.
- Review System: Authenticated users can submit and view reviews for specific Movie entries. Form submissions are validated using both client-side (JavaScript) and server-side (ModelState) validation.
- Navigation: A consistent navigation bar provides access to all major sections, including catalogs, search, and user account management.

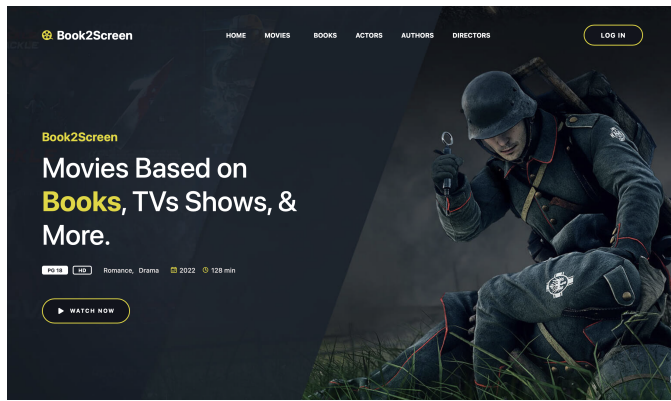


Figure 3: Landing page hero section with primary call-to-action and navigational hierarchy.

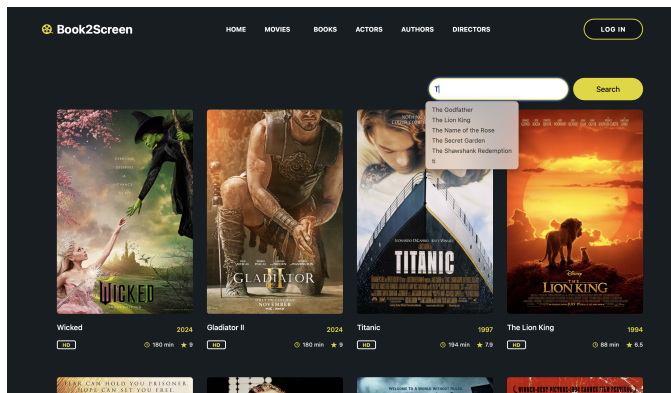


Figure 4: Movie catalog interface demonstrating the responsive card layout and integrated search bar.

V. DEPLOYMENT AND ACCESSIBILITY

The application is configured for a production environment. Continuous Deployment (CD) is established via GitHub integration with Azure App Service. Environment variables are managed within Azure for configuration security (e.g.,

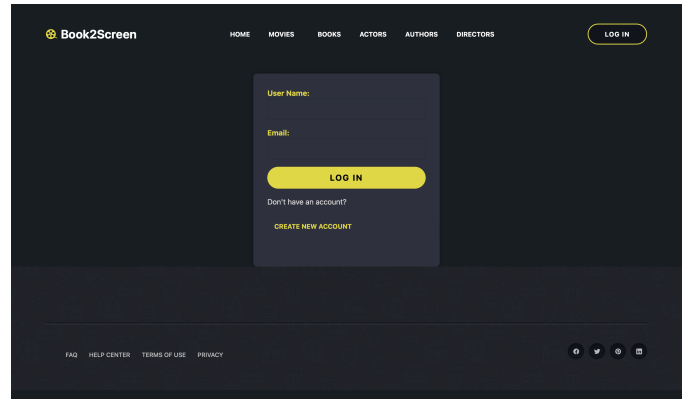


Figure 5: Secure authentication interface for user registration and login.

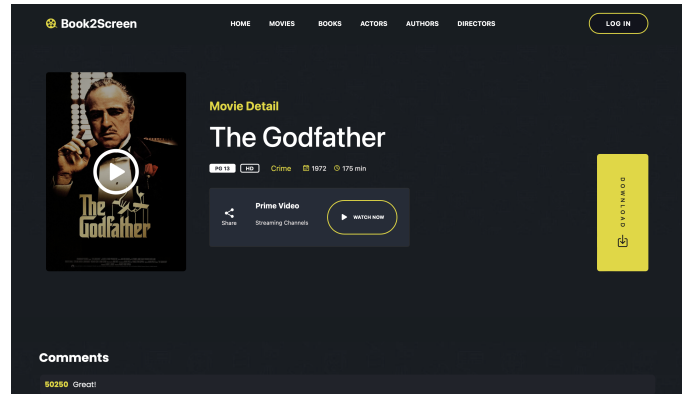


Figure 6: Detailed entity view showing comprehensive information about a specific adaptation.

database connection strings). The platform is publicly accessible at <https://book2screen.azurewebsites.net>, providing a live demonstration of the implemented features.

VI. CONCLUSION

Book2Screen successfully implements a functional, web-based platform that structurally connects literary works and their film adaptations. It validates the use of a relational database as an effective model for representing the complex, multidimensional relationships inherent in adaptation studies. The integration of a modern .NET backend with a responsive Bootstrap frontend, deployed on a scalable cloud infrastructure, demonstrates a complete full-stack development lifecycle.

ACKNOWLEDGMENT

I would like to thank the Faculty of Engineering and Architecture at Fenerbahçe University for their academic support and resources during this project development. Special appreciation is extended to Kokteyl Bilgi Teknolojileri for the practical experience and technical insights gained during my internship, which significantly contributed to the implementation of this application.

REFERENCES

- [1] Microsoft, “ASP.NET Core Documentation,” [Online]. Available: <https://docs.microsoft.com/en-us/aspnet/core/>.
- [2] The PostgreSQL Global Development Group, “PostgreSQL Documentation,” [Online]. Available: <https://www.postgresql.org/docs/>.
- [3] Bootstrap Team, “Bootstrap v5.3 Documentation,” [Online]. Available: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>.
- [4] J. D. Ullman and J. Widom, A First Course in Database Systems, 3rd ed. Pearson, 2007.
- [5] F. L. Hitchcock, “Adaptation in the Age of Digital Media: A Database Approach,” *Journal of Film and Literature Studies*, vol. 12, no. 3, pp. 45–67, 2023.
- [6] A. S. Tanenbaum and M. van Steen, Distributed Systems: Principles and Paradigms, 2nd ed. Pearson, 2006.
- [7] R. C. Martin, Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Prentice Hall, 2017.